Платформено-независими програмни езици

Лекция 10

Java и изкуствен интелект

Езикови модели от Java, LangChain4j и Spring AI, локални модели, невронни мрежи с ONNX и DJL, ИИ асистенти при програмирането

проф. д-р Даниела Гоцева
Учебна 2026/2027 г.



Съдържание на лекцията

1. Припомняне от първи курс
2. Java в света на ИИ
3. Ниво 1: езикови модели през API
4. LangChain4j и Spring AI
5. Ниво 2: локални модели
6. Ниво 3: невронни мрежи и Java
7. ИИ при програмирането на Java

01

Припомняне от първи курс

„Python и изкуствен интелект“ – основните понятия

ИИ, машинно обучение, дълбоко обучение

- **Изкуствен интелект** – системи, които изпълняват задачи, за които обикновено е нужен човешки интелект
- **Машинно обучение** – програмата се учи от **примери**, вместо правилата да се програмират ръчно
- **Дълбоко обучение** – машинно обучение с многослойни **невронни мрежи**
- **Генеративен ИИ** – модели, които създават нов текст, изображения, звук или код
- Всяко следващо понятие е част от предишното

Езиковият модел предсказва следващия токен

- Текстът се разделя на **токени** – думи или части от думи
- Моделът изчислява вероятност за всеки възможен следващ токен
- Избира един, добавя го и повтаря
- Обучение: предварително (огромно количество текст) → дообучение → обратна връзка от хора
- Примери: Claude (Anthropic), ChatGPT (OpenAI), Gemini (Google), Llama (Meta)

Ограничения, които трябва да познаваме

Ограничение	Какво означава
Халюцинации	измислени факти, функции или източници, казани уверено
Грешен или остарял код	изглежда правилно, но не работи или ползва стари версии на API
Различни отговори	един и същ въпрос може да даде различен отговор
Ограничени знания	моделът не знае събитията след датата на обучението си

Три нива на работа с ИИ

Ниво	Какво	Пример	Къде го учим
1	готов модел през API	Claude, ChatGPT, Gemini	I курс (Python), **днес – Java**
2	отворен модел локално	Llama, Mistral чрез Ollama	I курс (Python), **днес – Java**
3	собствен или дообучен модел	невронни мрежи, PyTorch	по-горните курсове; **днес – как Java го използва**

Помните ли? Заявка към езиков модел от Python

```python
# pip install anthropic
import anthropic

client = anthropic.Anthropic()        # ключът се чете от ANTHROPIC_API_KEY

response = client.messages.create(
    model="claude-sonnet-5-5",
    max_tokens=500,
    messages=[{"role": "user", "content": "Какво е речник в Python?"}]
)
print(response.content[0].text)
```

Днес ще направим същото от Java – първо „на ръка“ с HTTP, после с библиотеки



ИИ е инструмент. Отговорността остава при човека.

Преди да използвате резултат от ИИ, питайте се: вярно ли е, разбирам ли го и бих ли се подписал под него? Правилата за етика, лични данни и авторски права от първи курс важат и тук.

02

Java в света на ИИ

Защо и къде Java използва изкуствен интелект

Python и Java в проектите с ИИ

Python – „лабораторията“

- експерименти и анализ на данни
- обучение на модели (PyTorch, TensorFlow)
- научни изследвания
- бързи прототипи в Jupyter

Java – „производството“

- големи системи: банки, телекоми, търговия
- сървъри, работещи 24/7 под натоварване
- Android приложения
- **вграждане** на ИИ в съществуващи системи

Екосистемата: езикови модели

Библиотека	Предназначение
`java.net.http.HttpClient`	заявки към ИИ услуги без допълнителни библиотеки
официални SDK (Anthropic, OpenAI, Google)	удобен достъп до конкретен доставчик
LangChain4j	рамка за ИИ приложения: модели, памет, инструменти, RAG
Spring AI	ИИ в Spring Boot приложения
MCP Java SDK	свързване на модели с инструменти и данни

Екосистемата: машинно обучение

Библиотека	Предназначение
ONNX Runtime	изпълнение на готови модели във формат ONNX
DJL (Deep Java Library)	модели от PyTorch, TensorFlow, ONNX; зоопарк от готови модели
Deeplearning4j	дефиниране и обучение на невронни мрежи в Java
Tribuo	класическо машинно обучение (Oracle)
Weka	класически алгоритми, графичен интерфейс за експерименти

Типични задачи с ИИ в Java приложения

- **Чат асистент** в уеб приложение или вътрешна система
- **Обобщаване и класифициране** на документи, имейли, заявки към поддръжката
- **Извличане на данни** от свободен текст към обекти (`record`)
- **Търсене по смисъл** в собствени документи (RAG)
- **Разпознаване на изображения** с готов модел
- **Агенти** – моделът извиква Java методи, за да изпълни задача

03

Ниво 1: езикови модели през API

HTTP + JSON – знанията от лекции 7 и 8 в действие

Как изглежда заявката?

- HTTP **POST** към адреса на услугата:
  `https://api.anthropic.com/v1/messages`
- Заглавия (headers):
  - `x-api-key` – ключ за достъп
  - `anthropic-version` – версия на API
  - `content-type: application/json`
- Тяло – **JSON** с модела, максималния брой токени и съобщенията
- Отговор – JSON, текстът е в `content[0].text`
- Това е същото, което прави библиотеката в Python – „под капака“

Заявка с HttpClient

```
String apiKey = System.getenv("ANTHROPIC_API_KEY");       // никога в кода!
String body = """
        {
          "model": "claude-sonnet-5-5",
          "max_tokens": 500,
          "messages": [{"role": "user", "content": "Какво е record в Java?"}]
        }
        """;
HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create("https://api.anthropic.com/v1/messages"))
        .header("x-api-key", apiKey)
        .header("anthropic-version", "2023-06-01")
        .header("content-type", "application/json")
        .POST(HttpRequest.BodyPublishers.ofString(body))
        .build();

HttpResponse<String> response = HttpClient.newHttpClient()
        .send(request, HttpResponse.BodyHandlers.ofString());
```

Разбор на отговора с Jackson

```
// Отговорът (съкратено):
// {"id":"msg_...","role":"assistant",
//  "content":[{"type":"text","text":"record е ..."}],
//  "usage":{"input_tokens":15,"output_tokens":120}}

if (response.statusCode() != 200) {
    throw new IllegalStateException("Грешка " + response.statusCode()
            + ": " + response.body());
}

ObjectMapper mapper = new ObjectMapper();
JsonNode root = mapper.readTree(response.body());

String answer = root.path("content").get(0).path("text").asText();
int outTokens = root.path("usage").path("output_tokens").asInt();

System.out.println(answer);
System.out.println("Изходни токени: " + outTokens);
```

Сглобяване на JSON безопасно

- Не слепвайте JSON с `+` – кавички и нови редове в текста ще го развалят
- Изграждаме обекти и ги сериализираме с Jackson
- Записите (`record`) се превръщат в JSON автоматично

```
record Message(String role, String content) { }

record ChatRequest(String model,
                   int max_tokens,
                   List<Message> messages) { }

var req = new ChatRequest(
        "claude-sonnet-5-5", 500,
        List.of(new Message("user",
                "Обясни \"var\" в Java")));

String body = new ObjectMapper()
        .writeValueAsString(req);
```

Ollama чрез HttpClient – без библиотека

- Локалният сървър на Ollama приема JSON на порт 11434
- Същата идея като при облачните API
- `"stream": false` – целият отговор наведнъж

```
String body = """
    {"model": "llama3.2",
     "stream": false,
     "messages": [{"role": "user",
       "content": "Какво е JVM?"}]}
    """;

var req = HttpRequest.newBuilder()
    .uri(URI.create(
        "http://localhost:11434/api/chat"))
    .POST(HttpRequest.BodyPublishers
            .ofString(body))
    .build();

var resp = HttpClient.newHttpClient().send(req,
    HttpResponse.BodyHandlers.ofString());
// отговорът е в message.content
```

Разговор с памет

- Моделът **не помни** предишните заявки
- Пазим историята в списък и я изпращаме всеки път
- Същата идея като в Python: списък от речници → тук списък от записи

```
List<Message> history = new ArrayList<>();
Scanner in = new Scanner(System.in);

while (true) {
    System.out.print("Вие: ");
    String q = in.nextLine();
    if (q.equals("край")) break;

    history.add(new Message("user", q));
    String answer = ask(history);   // HTTP
    history.add(
        new Message("assistant", answer));

    System.out.println("ИИ: " + answer);
}
```

Грешки и ограничения при работа с API

Ситуация	HTTP код	Какво правим
невалиден или липсващ ключ	401	проверяваме променливата на средата
грешна заявка (JSON, модел)	400	четем съобщението в тялото на отговора
твърде много заявки	429	изчакваме и опитваме отново (с нарастваща пауза)
претоварена услуга	5xx	повторен опит, резервен вариант
бавен отговор	–	`timeout` на заявката, стрийминг
цена	–	следим `usage` – плаща се на токен

Цена и производителност

- Плаща се за **входни и изходни токени** – дългата история струва повече
- `max_tokens` – ограничава дължината на отговора
- Кеширайте отговори на еднакви въпроси (като речника от 1. курс)
- Избирайте по-малък модел, когато е достатъчен
- **Стрийминг** – текстът пристига на части, потребителят вижда отговора веднага
- Задавайте `timeout` и обработвайте повторните опити

Официален Java SDK

- Доставчиците предлагат готови библиотеки
- Скриват HTTP и JSON, дават типове и builder-и
- Anthropic: `com.anthropic:anthropic-java`
- OpenAI и Google имат аналогични SDK

```
AnthropicClient client =
        AnthropicOkHttpClient.fromEnv();

MessageCreateParams params =
    MessageCreateParams.builder()
        .model("claude-sonnet-5-5")
        .maxTokens(500L)
        .addUserMessage("Какво е JVM?")
        .build();

Message msg = client.messages()
        .create(params);

msg.content().forEach(block ->
        block.text().ifPresent(t ->
                System.out.println(t.text())));
```



API ключът е парола – никога не го пишете в кода

Използвайте променлива на средата (`System.getenv`) или конфигурационен файл извън Git. Ключ, качен в GitHub, се открива от ботове за минути.

04

LangChain4j и Spring AI

Рамки за ИИ приложения

Какво добавя една рамка?

- **Единен интерфейс** към много модели – смяна на доставчика с един ред
- **AI Services** – описваме Java интерфейс, рамката го реализира чрез модела
- **Памет** за разговора
- **Структуриран изход** – отговорът директно като Java обект
- **Инструменти** (tools) – моделът може да извиква наши Java методи
- **RAG** – отговори въз основа на собствени документи
- **MCP** – стандартен протокол за свързване с външни инструменти

LangChain4j – първа стъпка

- Maven зависимост: `dev.langchain4j:langchain4j-anthropic`
- За OpenAI, Gemini, Ollama – съответният модул
- `ChatModel` – общ интерфейс за всички модели

```
ChatModel model = AnthropicChatModel.builder()
        .apiKey(System.getenv(
                "ANTHROPIC_API_KEY"))
        .modelName("claude-sonnet-5-5")
        .maxTokens(500)
        .build();

String answer = model.chat(
        "Обясни полиморфизма с един пример");

System.out.println(answer);
```

AI Services – интерфейс вместо код

- Описваме **какво** искаме като Java интерфейс
- `@SystemMessage` – роля и правила за модела
- `@UserMessage` с `{{променлива}}` – шаблон на заявката
- LangChain4j създава реализацията

```
interface Tutor {
    @SystemMessage("""
        Ти си асистент по Java за студенти
        от втори курс. Отговаряй кратко,
        на български, с пример.""")
    String explain(String topic);

    @UserMessage("Преведи на {{lang}}: {{text}}")
    String translate(@V("text") String text,
                     @V("lang") String lang);
}

Tutor tutor = AiServices.create(
        Tutor.class, model);
tutor.explain("ламбда изрази");
```

Памет за разговора

- `MessageWindowChatMemory` – пази последните N съобщения
- Историята се изпраща автоматично
- За много потребители – отделна памет за всеки (`@MemoryId`)

```
interface Assistant {
    String chat(String message);
}

Assistant a = AiServices
        .builder(Assistant.class)
        .chatModel(model)
        .chatMemory(MessageWindowChatMemory
                .withMaxMessages(20))
        .build();

a.chat("Казвам се Мария.");
a.chat("Как се казвам?");
// "Казвате се Мария."
```

Структуриран изход

- Методът връща **запис** вместо текст
- Рамката описва на модела очаквания формат и разбира JSON отговора
- Извличане на данни от свободен текст

```
record Invoice(String company,
               LocalDate date,
               double total,
               String currency) { }

interface Extractor {
    @UserMessage("Извлечи данните: {{it}}")
    Invoice extract(String text);
}

Extractor ex = AiServices.create(
        Extractor.class, model);

Invoice inv = ex.extract("""
    Фактура от ТехноСофт ООД, издадена на
    3 март 2026 г., на стойност 1250,50 евро.
    """);
// Invoice[company=ТехноСофт ООД, ...]
```

Инструменти (tools) – моделът вика Java

- Методи с `@Tool` – описание за модела
- Моделът решава **кога** да извика инструмента и с какви аргументи
- LangChain4j изпълнява метода и връща резултата на модела
- Основата на **ИИ агентите**

```
class Bank {
    @Tool("Връща баланса на сметка по IBAN")
    double balance(String iban) {
        return repository.find(iban)
                         .getBalance();
    }

    @Tool("Курс на валута спрямо евро")
    double rate(String currency) {
        return rates.get(currency);
    }
}

Assistant a = AiServices
        .builder(Assistant.class)
        .chatModel(model)
        .tools(new Bank())
        .build();
```

RAG – отговори от собствени документи

- Моделът не знае вашите документи: правилник, учебни материали, документация
- **RAG** (Retrieval-Augmented Generation):
  - 1. Документите се разделят на части и се превръщат във **вектори** (embeddings)
  - 2. Векторите се пазят във **векторна база**
  - 3. При въпрос се намират най-близките по смисъл части
  - 4. Те се добавят към заявката като контекст
- Моделът отговаря въз основа на тях – по-малко халюцинации, посочени източници

RAG с LangChain4j

```java
// 1. Зареждане и индексиране на документите
List<Document> docs = FileSystemDocumentLoader.loadDocuments(Path.of("docs"));
InMemoryEmbeddingStore<TextSegment> store = new InMemoryEmbeddingStore<>();
EmbeddingStoreIngestor.ingest(docs, store);

// 2. Асистент, който търси в документите
interface DocsAssistant {
    String answer(String question);
}

DocsAssistant assistant = AiServices.builder(DocsAssistant.class)
        .chatModel(model)
        .contentRetriever(EmbeddingStoreContentRetriever.from(store))
        .build();

assistant.answer("Колко пъти може да се явя на изпит по Java?");
```

Пример с „лесния“ RAG модул на LangChain4j; в реални системи – векторна база (PostgreSQL + pgvector и др.)

Embeddings – вектори на смисъла

- **Embedding** – текстът се превръща във вектор от стотици числа
- Текстове с близък смисъл имат **близки** вектори
  - „котка“ е близо до „коте“, далеч от „фактура“
- Близост – косинусово подобие (ъгъл между векторите)
- Основата на RAG, семантичното търсене, препоръките
- Векторни бази: PostgreSQL + pgvector, Chroma, Qdrant

MCP – Model Context Protocol

- Отворен стандарт за свързване на ИИ модели с **инструменти и данни**
- **MCP сървър** предоставя инструменти: файлове, база данни, GitHub, календар…
- **MCP клиент** (асистент, приложение) ги използва по единен начин
- Комуникация с JSON съобщения – през стандартния вход/изход или HTTP
- Java: официален MCP Java SDK; поддръжка в LangChain4j и Spring AI
- Пример: Java приложение предоставя данните си като MCP сървър → всеки ИИ асистент може да ги ползва

Spring AI

- За приложения със **Spring Boot**
- Spring AI 2.0 (2026) – за Spring Boot 4
- `ChatClient` – верижен (fluent) API
- Конфигурация в `application.properties`
- `entity(...)` – структуриран изход

```
@RestController
class TutorController {
    private final ChatClient chat;

    TutorController(ChatClient.Builder b) {
        this.chat = b.defaultSystem(
            "Ти си асистент по Java.").build();
    }

    @GetMapping("/ask")
    String ask(@RequestParam String q) {
        return chat.prompt()
                   .user(q)
                   .call()
                   .content();
    }
}
```

Тестване на код с ИИ: fake модел

- Бизнес логиката зависи от интерфейса `ChatModel`
- В теста – фиксиран отговор, без мрежа и без пари
- Тестът е бърз и повторим

```
interface Summarizer {
    String summarize(String text);
}

record AiSummarizer(ChatModel model)
        implements Summarizer {
    public String summarize(String text) {
        return model.chat("Обобщи: " + text);
    }
}

@Test
void usesModelAnswer() {
    ChatModel fake = msg -> "кратко";
    var s = new AiSummarizer(fake);
    assertEquals("кратко", s.summarize("..."));
}
```

05

Ниво 2: локални модели

Отворени модели на собствения компютър с Ollama

Защо локален модел?

- **Безплатно** – без ключ и без плащане на токен
- **Поверителност** – данните не напускат компютъра
- **Без интернет** след изтеглянето на модела
- Компромис: по-малки модели, по-ниско качество, нужни са памет и процесор
- **Ollama**: `ollama pull llama3.2` – модел; работи като HTTP сървър на порт 11434
- От Python в първи курс: `ollama.chat(...)` – сега от Java

Ollama от Java

- Същият код с LangChain4j – сменя се само моделът
- Модул: `dev.langchain4j:langchain4j-ollama`
- Или директно с `HttpClient` към `http://localhost:11434/api/chat`

```
ChatModel local = OllamaChatModel.builder()
        .baseUrl("http://localhost:11434")
        .modelName("llama3.2")
        .temperature(0.3)
        .build();

Tutor tutor = AiServices.create(
        Tutor.class, local);

System.out.println(
        tutor.explain("интерфейси"));
```



Програмирайте към интерфейса – и при ИИ

Кодът зависи от `ChatModel`, а не от конкретен доставчик. Облачен модел в продукцията, локален при разработката, фиксиран „fake“ модел в тестовете – без промяна на бизнес логиката.

06

Ниво 3: невронни мрежи и Java

Обучение в Python, използване в Java

Как моделът стига от Python до Java?

- **1. Обучение в Python** – PyTorch, TensorFlow/Keras, scikit-learn
- **2. Експорт във файл** – най-често **ONNX** (`model.onnx`)
  - отворен формат, независим от езика
  - съдържа архитектурата на мрежата и обучените тегла
- **3. Използване в Java** – ONNX Runtime или DJL
  - само **предсказване** (inference), без обучение
- Алтернатива: моделът остава на Python сървър, а Java го вика по HTTP

Обучение на малка мрежа в Python (PyTorch)

```
import torch, torch.nn as nn
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)          # 150 цвята, 4 признака, 3 вида
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y)

model = nn.Sequential(nn.Linear(4, 16), nn.ReLU(), nn.Linear(16, 3))
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.Adam(model.parameters(), lr=0.01)

for epoch in range(200):                   # обучение
    opt.zero_grad()
    loss = loss_fn(model(X), y)
    loss.backward()
    opt.step()
```

Мрежа с 4 входа, скрит слой от 16 неврона и 3 изхода – по един за всеки вид ирис

Експорт на модела в ONNX

- Една функция записва обучената мрежа във файл
- Нужен е примерен вход – за да се определят размерите
- Именуваме входа и изхода – по тези имена ги ползва Java
- Файлът `iris.onnx` е готов за всеки език: Java, C#, C++, JavaScript

```
# pip install torch scikit-learn onnx onnxscript

model.eval()                    # режим "предсказване"

torch.onnx.export(
    model,
    torch.randn(1, 4),          # примерен вход
    "iris.onnx",
    input_names=["input"],
    output_names=["logits"])
```

Използване на модела в Java (ONNX Runtime)

```
// Maven: com.microsoft.onnxruntime:onnxruntime
String[] species = {"setosa", "versicolor", "virginica"};
var env = OrtEnvironment.getEnvironment();
float[][] flower = {{5.1f, 3.5f, 1.4f, 0.2f}};       // размери на цвета

try (var session = env.createSession("iris.onnx");
     var input = OnnxTensor.createTensor(env, flower);
     var result = session.run(Map.of("input", input))) {

    float[] logits = ((float[][]) result.get(0).getValue())[0];
    int best = 0;
    for (int i = 1; i < logits.length; i++) {
        if (logits[i] > logits[best]) best = i;
    }
    System.out.println("Вид: " + species[best]);      // setosa
}
```

DJL – готови модели

- **Deep Java Library** от Amazon
- Изтегля готови модели от „зоопарк“ на модели
- Работи с PyTorch, TensorFlow, ONNX
- Пример: класифициране на изображение

```
Criteria<Image, Classifications> criteria =
    Criteria.builder()
        .setTypes(Image.class,
                  Classifications.class)
        .optApplication(Application.CV
                .IMAGE_CLASSIFICATION)
        .build();

try (var model = criteria.loadModel();
     var predictor = model.newPredictor()) {
    Image img = ImageFactory.getInstance()
            .fromFile(Path.of("cat.jpg"));
    System.out.println(
            predictor.predict(img).best());
}
```

Невронна мрежа – какво има вътре?

- **Неврон**: претеглена сума на входовете + отместване → активираща функция
  - `y = f(w1·x1 + w2·x2 + … + b)`
- **Слой** – много неврони; **мрежа** – няколко слоя
- **Обучение** – теглата `w` и `b` се настройват, така че грешката да намалява
- Най-простият случай: **перцептрон** – един неврон, две класи
- Изчисленията са умножения на матрици – затова се използват GPU

Перцептрон на чиста Java

```
class Perceptron {
    private final double[] w;          // тегла
    private double b;                  // отместване
    private final double rate = 0.1;   // скорост на обучение

    Perceptron(int n) { w = new double[n]; }

    int predict(double[] x) {
        double sum = b;
        for (int i = 0; i < w.length; i++) sum += w[i] * x[i];
        return sum >= 0 ? 1 : 0;       // стъпаловидна активация
    }

    void learn(double[] x, int y) {      // една стъпка на обучение
        int err = y - predict(x);        // 0, 1 или -1
        for (int i = 0; i < w.length; i++) w[i] += rate * err * x[i];
        b += rate * err;
    }
}
```

Перцептронът научава логическо И

- Данни: четирите комбинации на два входа
- След обучението теглата са настроени
- Перцептронът научава **И** и **ИЛИ**, но не и **XOR** – за него са нужни повече слоеве

```
double[][] xs = {{0, 0}, {0, 1},
                 {1, 0}, {1, 1}};
int[] and = {0, 0, 0, 1};

Perceptron p = new Perceptron(2);
for (int e = 0; e < 20; e++)
    for (int k = 0; k < xs.length; k++)
        p.learn(xs[k], and[k]);

for (double[] x : xs)
    System.out.println(Arrays.toString(x)
            + " -> " + p.predict(x));
// [0.0, 0.0] -> 0
// [0.0, 1.0] -> 0
// [1.0, 0.0] -> 0
// [1.0, 1.0] -> 1
```

Обучение на мрежи изцяло в Java

- **Deeplearning4j** (Eclipse Foundation) – слоеве, обучение и оценка в Java; импорт на модели от Keras
- **Tribuo** (Oracle) – класификация, регресия, клъстеризация
- **Weka** – класически алгоритми; има и графичен интерфейс
- В практиката обучението най-често е в Python, а Java **използва** готовите модели
- Подробно – в курсовете по машинно обучение в трети и четвърти курс

07

ИИ при програмирането на Java

Асистенти, проверка на кода, отговорност

ИИ асистенти за програмисти

Инструмент	Какво прави	Среда
GitHub Copilot	допълване на код, чат, промени в няколко файла	Eclipse, IntelliJ, VS Code
Claude Code	агент: чете проекта, пише код, пуска тестове	терминал, IDE
JetBrains AI Assistant	чат и генериране в IntelliJ IDEA	IntelliJ
чат асистенти	обяснения, идеи, преглед на код	браузър

Работен процес с ИИ асистент

- 1. **Вие** формулирате задачата ясно: контекст, ограничения, пример
- 2. **Вие** пишете тестовете (лекция 9) – те описват очакваното поведение
- 3. Асистентът предлага код
- 4. **Четете** всеки ред – разбирате ли го?
- 5. Пускате тестовете – червено → обратно към асистента или поправка от вас
- 6. Проверявате версии на библиотеки и API в документацията
- 7. Посочвате, че сте използвали ИИ, когато това се изисква

Кога ИИ помага и кога пречи на ученето

Помага

- обяснение на понятие с друг пример
- разбиране на съобщение за грешка
- преглед на код, който **вие** сте написали
- идеи за тестови случаи
- превод на код от Python към Java

Пречи

- готово решение, което не разбирате
- пропускане на собствения опит
- сляпо доверие в отговора
- използване там, където не е разрешено
- код, който не можете да обясните на изпита

Припомняне: Акт на ЕС за ИИ

Ниво на риск	Примери	Какво означава за разработчика
Забранен	социално точкуване, манипулация	такива системи не се разработват
Висок риск	наемане, кредити, образование	документация, оценка на риска, човешки надзор
Прозрачност	чатботове, генерирано съдържание	потребителят трябва да знае, че общува с ИИ
Минимален риск	спам филтри, игри	без допълнителни изисквания

Обобщение

- Три нива: модел през API, локален модел, собствен модел
- API = HTTP POST + JSON – `HttpClient` и Jackson от предишните лекции
- Официални SDK и рамките **LangChain4j** и **Spring AI**: AI Services, памет, структуриран изход, инструменти, RAG, MCP
- Локални модели с **Ollama** – същият код, друг `ChatModel`
- Невронни мрежи: обучение в Python → **ONNX** → използване в Java (ONNX Runtime, DJL)
- Перцептронът показва какво има в един неврон
- ИИ асистентите помагат, но тестовете и отговорността са ваши

Въпроси за самопроверка

- Кои са трите нива на работа с ИИ? Дайте пример за всяко.
- Как изглежда HTTP заявката към езиков модел? Къде се пази ключът?
- Защо моделът „не помни“ разговора и как се решава това?
- Какво е AI Service в LangChain4j? Какво е инструмент (tool)?
- Какво е RAG и какъв проблем решава?
- Как модел, обучен в Python, се използва от Java?
- Какво може и какво не може да научи един перцептрон?

Въпроси?

Благодаря за вниманието!

